

Algorithm III – Week 3 Lesson Plan

Structured Records, Modular Functions, and Text-File Processing

Lesson Objectives

Knowledge

- Explain how a C struct groups related fields into one record and how an array of structs represents a record collection.
- Distinguish passing a record by value, passing its address for modification, and passing a read-only pointer.
- Describe a simple text-file format and the stages open, read, validate, process, save, close.
- Explain how search, update, and sort operate on logical records rather than unused capacity.
- Identify malformed-input, duplicate-key, file-open, partial-read, and save-failure cases.

Skills

- Define, initialize, access, copy, print, compare, and update structured records.
- Write modular C functions for input, display, search, update, sorting, loading, and saving.
- Load validated records into a bounded or dynamic collection without corrupting accepted data.
- Sort records by a selected field while moving complete records together.
- Test the record manager with normal, empty, boundary, duplicate, and malformed files.

Attitude

- Treat a file as untrusted input that must be checked before a record is committed.
- Preserve record identity and field relationships during every operation.
- Prefer small functions with explicit success/failure results and observable test evidence.

Lesson Structure (3 Hours)

Part 1: Week 2 Bridge—From Parallel Arrays to Records (15 mins)

```
int ids[3] = {101, 102, 103};
char names[3][30] = {"Dara", "Sokha", "Maly"};
float scores[3] = {78.5f, 91.0f, 84.0f};
```

- Which three values describe one student?
- What error occurs if only one parallel array is shifted or sorted?
- How can one variable keep the fields together?

Key bridge: Week 2 managed a dynamic collection of integers. Week 3 changes each element into a complete record and adds persistent text-file input/output.

Part 2: Defining and Accessing a Record (20 mins)

```
typedef struct {
    int id;
    char name[30];
    float score;
} Student;

Student s = {101, "Dara", 78.5f};
printf("%d %s %.1f\n", s.id, s.name, s.score);
```

Expression	Meaning	Use
s.field	Access a field through a record object	s.score
p->field	Access a field through a pointer	p->score
records[i]	Select one complete record	records[2]
records[i].field	Select one field in one record	records[i].id

- A struct assignment copies every field in the record.
- Character arrays require bounded input; they are not assigned with = after declaration.
- The record key (here id) should remain unique if the application depends on unique lookup.

Part 3: Passing Records to Modular Functions (20 mins)

```
void printStudent(const Student *s) {
    printf("%d | %s | %.1f\n", s->id, s->name, s->score);
}

int updateScore(Student *s, float newScore) {
    if (newScore < 0 || newScore > 100) return 0;
    s->score = newScore;
    return 1;
}
```

- Use `const Student *` when the function should observe but not modify the record.
- Use `Student *` when a successful function must update the caller's record.
- Return a status so invalid updates are visible and leave the old record unchanged.
- Keep one responsibility per function: validation, search, display, update, load, save, or sort.

Part 4: Arrays of Records and Search (20 mins)

```
int findById(const Student records[], int size, int targetId) {
    for (int i = 0; i < size; i++) {
        if (records[i].id == targetId) return i;
    }
    return -1;
}
```

- Search only indexes below logical size.
- Return the matching index so another function can display or update the record.

- Before adding a new record, search for its ID to enforce uniqueness.
- An empty collection safely returns -1 without examining a record.

Part 5: Text-File Format and the Read–Validate–Commit Pattern (30 mins)

Use one simple, documented format for the lesson. Example file: students.txt

```
101|Dara|78.5
102|Sokha|91.0
103|Maly|84.0

char line[128], name[30], extra;
int id;
float score;
while (fgets(line, sizeof line, fp) != NULL) {
    if (sscanf(line, "%d|%29[^\n]|%f %c",
               &id, name, &score, &extra) != 3) {
        /* reject malformed line */
        continue;
    }
    /* validate range and duplicate ID, then commit */
}
```

- Read one complete line with fgets so a bad line can be rejected as one unit.
- Parse into temporary variables; do not write directly into the collection.
- Validate field count, ID, score range, name length/content, duplicates, and available capacity.
- Commit a record and increment size only after every check succeeds.
- Track line number, accepted count, and rejected count for an auditable result.

Part 6: Opening, Loading, Saving, and Closing (20 mins)

```
FILE *fp = fopen("students.txt", "r");
if (fp == NULL) return 0;
/* read, parse, validate, commit */
if (fclose(fp) != 0) return 0;
```

Mode	Purpose	Important effect
r	Read existing records	Fails if file does not exist
w	Write a complete replacement	Truncates an existing file
a	Append new text	Writes at the end; may bypass full-file validation

- Check fopen before any read or write.
- Check fprintf and fclose when saving important data; never claim success after a write or close failure. For safer replacement, finish a temporary file before replacing the original.

Part 7: Updating and Sorting Complete Records (20 mins)

```
for (int pass = 0; pass < size - 1; pass++) {
    for (int i = 0; i < size - 1 - pass; i++) {
        if (records[i].score < records[i + 1].score) {
            Student temp = records[i];
            records[i] = records[i + 1];
            records[i + 1] = temp;
        }
    }
}
```

- Search by ID, validate the replacement value, then update the located record.
- Swap complete Student objects; never swap only scores or names.
- State the chosen sort direction and how ties are handled.
- Sorting changes in-memory order; persistence changes only after a successful save.

Part 8: Integrated File-Based Record Manager Trace (20 mins)

Step	State/result	Correctness check
Open file	Read handle or failure	Do not continue with NULL
Read line 1	101, Dara, 78.5 accepted	3 fields; valid range; unique ID
Read malformed line	Rejected; size unchanged	Report source line
Search 101	Index 0	Search only accepted records
Update score	101 becomes 82.0	Validate before commit
Sort descending	Whole records move	IDs remain attached to names/scores
Save	All accepted current records written	Check write and close results

Part 9: Boundary Testing and Exit Check (15 mins)

- Missing file, empty file, one valid record, maximum accepted records, extra records beyond capacity.
- Blank line, missing field, extra field, nonnumeric ID/score, long name, out-of-range score.
- Duplicate ID, absent search key, first/last record update, tied sort keys.
- Output-open failure, write failure, and preservation of the original data when a safe save fails.

Student Explanation Sequence

1. What fields define one record, and which field is its key?
2. Is the function reading a record, modifying it, or modifying the collection?
3. What input format and success condition are expected?

4. Which temporary values are parsed before commitment?
5. Which validations must pass, and what remains unchanged on failure?
6. Does the operation move or update the complete record?
7. How are file, boundary, malformed, duplicate, and save-failure cases tested?

Evaluation Questions

- Why is one Student record safer than three parallel arrays?
- When should a parameter be `const Student *` rather than `Student *`?
- Why should a search function return an index instead of only true/false?
- Why should a file line be parsed into temporary variables?
- When may size be increased during loading?
- How should a malformed line affect previously accepted records?
- Why must duplicate keys be detected before commitment?
- What does `fopen` returning `NULL` mean?
- Why is `w` mode potentially destructive?
- Why must sorting swap complete records?
- Does sorting automatically update the file? Explain.
- Which test cases demonstrate safe loading, updating, sorting, and saving?

Teacher Checklist

- Student identifies object field access (`.`) and pointer field access (`->`).
- Student uses `const` for read-only record parameters and pointers for intended updates.
- Student searches and processes only logical records.
- Student reads a whole line, parses temporary values, validates, then commits.
- Student rejects malformed and duplicate records without changing accepted data.
- Student swaps whole records and states sort order.
- Student checks open, write, and close results and explains safe replacement.
- Student demonstrates normal, empty, boundary, invalid, and failure tests.

Teaching Notes and Source Alignment

- Keep implementation in C using `typedef struct`, `FILE *`, `fopen`, `fgets`, `sscanf`, `fprintf`, and `fclose`.
- Use a small Student record so every operation can be traced visibly.
- Use a bounded record array first; optionally connect it to Week 2 dynamic resizing during the hard practice task.
- Tony Gaddis, *Starting Out with C++*: use the structured-data, arrays, searching, sorting, and file-processing discussions as conceptual support; implement the lesson with C facilities.
- The mission supplies reusable fragments and reasoning checkpoints, not a complete final record-manager program.